Full Stack Development with AI
Lab 16.2 – Hands-on MLflow

Lab Overview

Recall that MLflow is an open-source platform for managing the lifecycle of machine learning projects. In particular, MLflow supports machine learning experimentation, reproducibility, deployment, and model management. In this lab, you will learn how to work with MLflow using scikit-learn models trained with the Iris flowers dataset.

Before proceeding, please ensure that you have the latest version of Python 3 installed on your computer. You may use either Windows, macOS or Linux as the lab instructions are independent of operating system.

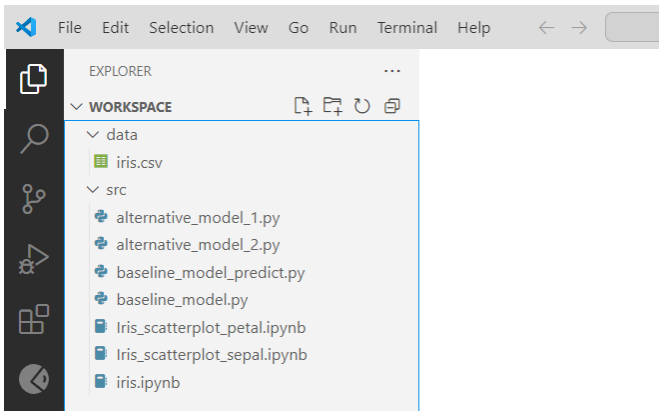
The resources that you need for this lab can be found in "workspace.zip".

Exercise 1 – Creating a Python Virtual Environment

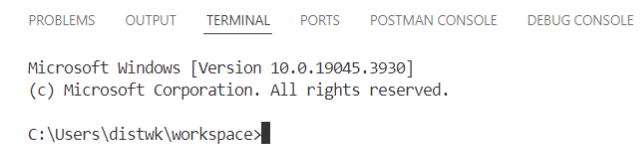
1. Extract "workspace.zip" on your local computer to a suitable location such as your home directory. On a Windows computer, this will be `C:\Users\<username>`. On a macOS and Linux computer, this will be `/Users/<username>`.

2. Start your Visual Studio Code (VS Code) and open the "workspace" folder that you have extracted in step (1).

   The Explorer window of your VS Code should resemble the screenshot below:



3. Start a new terminal window in VS Code and verify that the current working directory is the "workspace" folder.



4. Run the following command to create a new Python 3 virtual environment within the "workspace" folder. If the `python` command does not work, try the `python3` command instead.

```
python -m venv .venv
```

5. Follow the instructions to activate the newly created virtual environment:

For Windows command prompt:

```
.venv\Scripts\activate.bat
```

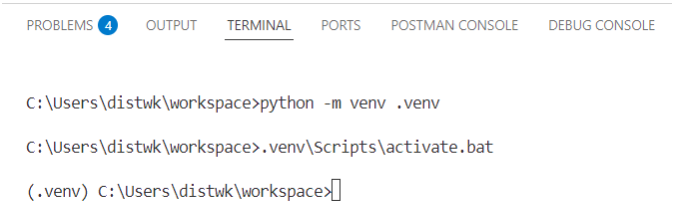
For Windows PowerShell:

```
.venv\Scripts\Activate.psl
```

For Linux and macOS:

```
source .venv/bin/activate
```

Your terminal should now resemble the following screenshot:

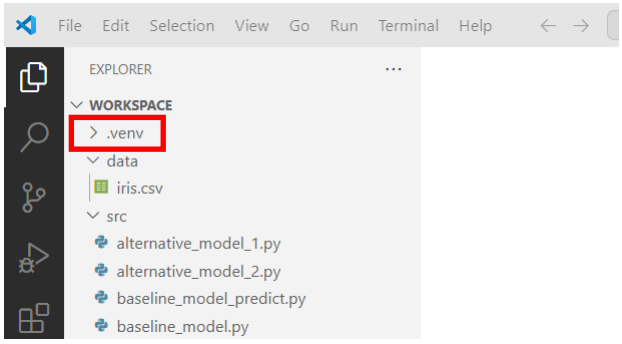


6. Run the following `pip` commands to install some Python libraries for machine learning:

```
pip install numpy
pip install pandas
pip install matplotlib
pip install scikit-learn
```

7. To exit from the virtual environment, use the `deactivate` command. But for now, please remain in the virtual environment.

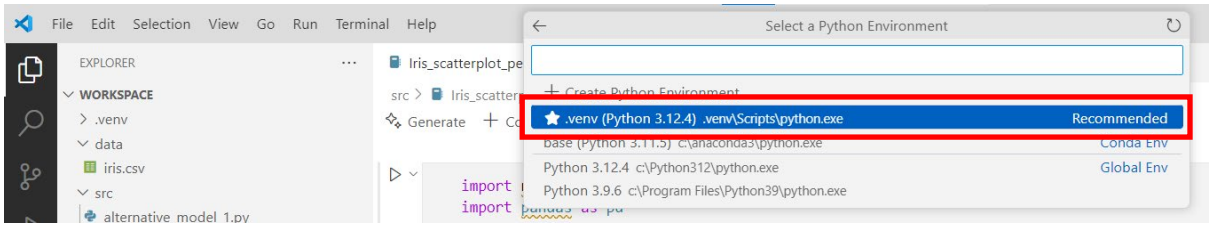
8. Observe that the Explorer window in your VS Code should show the new ".venv" folder.



9. Open "src\Iris_scatterplot_petal.ipynb" in your VS Code. Then click on the "Select Kernel" button.



10. Select "Python Environments" and you should see the newly created virtual environment as the recommended option. Select that option.



11. Run the Notebook and observe the output. When prompted to install "ipykernel", proceed to do so. Ensure that you are able to see the scatter plot of petal length and petal width.

12. Close the Notebook when you are done.

Exercise 2 – Installing MLflow

MLflow (https://www.mlflow.org/) is based on Python and thus it is relatively easy to install and set up. In fact, the easiest way to install MLflow is to do so from PyPi using `pip`.

Please ensure that your terminal window is in the Python virtual environment. If you have deactivated the virtual environment or started a new terminal window, do remember to activate the virtual environment before proceeding.

Follow the instructions below to install and set up MLflow on your local computer:

1. Run the following `pip` commands to install MLflow

```
pip install mlflow
```

Wait for the installation to complete.

2. Run the following command to verify that MLflow has been installed successfully:

```
python -m mlflow --version
```

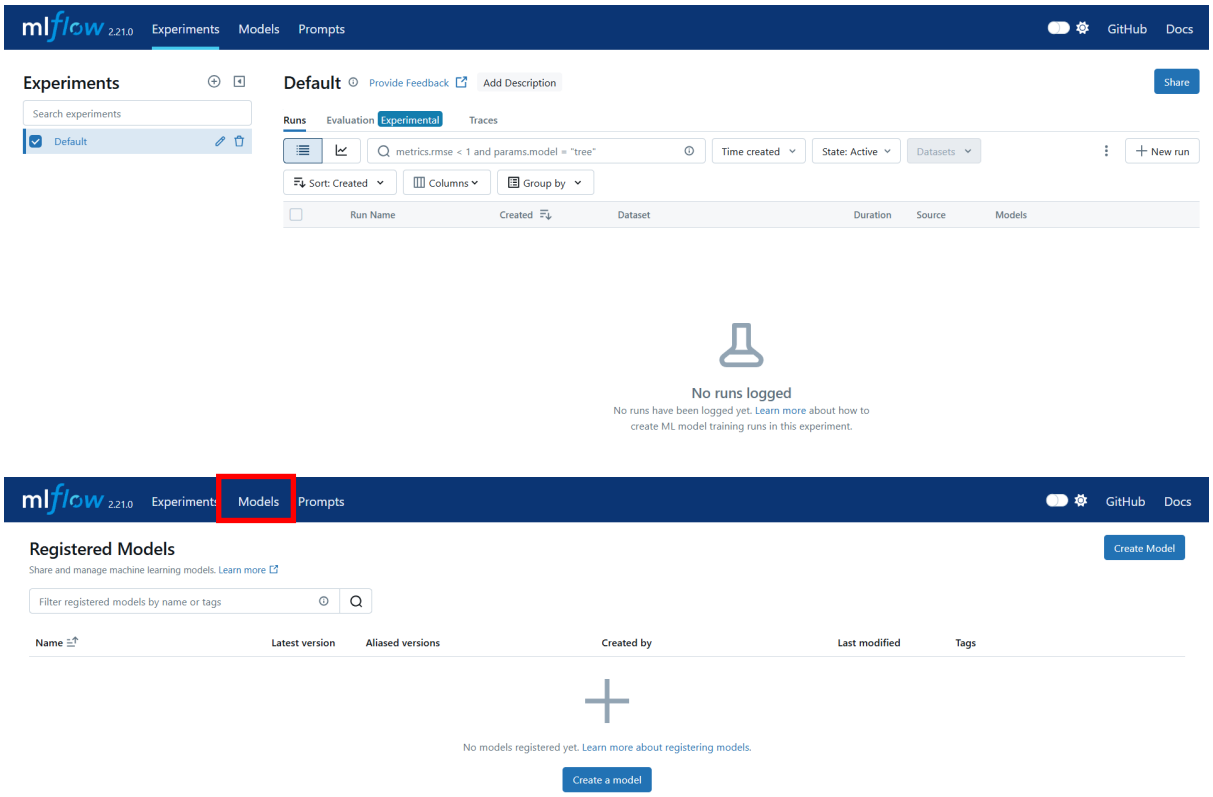
```
PROBLEMS  OUTPUT  TERMINAL  PORTS  JUPYTER  POSTMAN CONSOLE  DEBUG CONSOLE

(.venv) C:\Users\distwk\workspace>python -m mlflow --version
python -m mlflow, version 2.22.0
```

3. Run the following command to start a local MLflow Tracking Server. You can replace the port number as desired. Note that the default port number is 8080. The following command will override the port number to 9080, which is a less commonly used port number compared to 8080.

```
python -m mlflow server --host 127.0.0.1 --port 9080
```

4. Open your web browser and navigate to http://localhost:9080 and verify that you are able to view the MLflow user interface. Observe that there is currently no experiment and model deployed on the MLflow Tracking Server.



5. To stop the Tracking Server, press Ctrl+C in terminal. But for now, please leave it running.

Exercise 3 – Managing Models and Experiments

In this exercise, you will be using the Iris flowers dataset to train a few decision tree classifier models to understand how to work with MLflow.

We will begin with a single baseline model before progressing to the more advanced use case of multiple models comparison. The models will be varied using different features and parameters. But essentially, you will be training a model and preparing the metadata for logging.

1. Start a local MLflow Tracking Server if you have stopped the server after installation:

```
python -m mlflow server --host 127.0.0.1 --port 9080
```

2. Start a new terminal window and activate the Python virtual environment. Remember not to close the terminal window that is running MLflow.

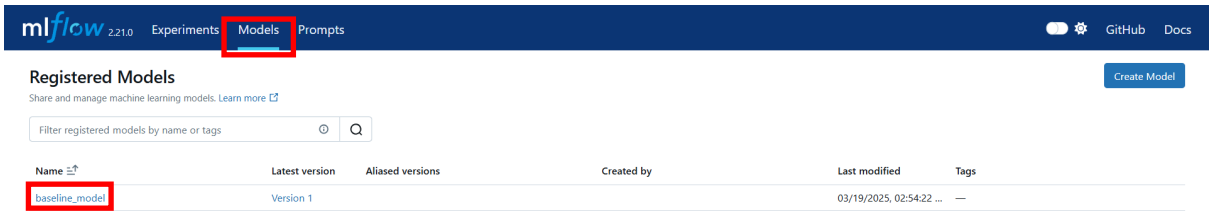
3. In the new terminal window, run "baseline_model.py". If your MLflow Tracking Server is started on a different port number from 9080, please amend line 38 in the code accordingly. You would also need to change the port number for all subsequent source files.

```
cd src
python baseline_model.py
```

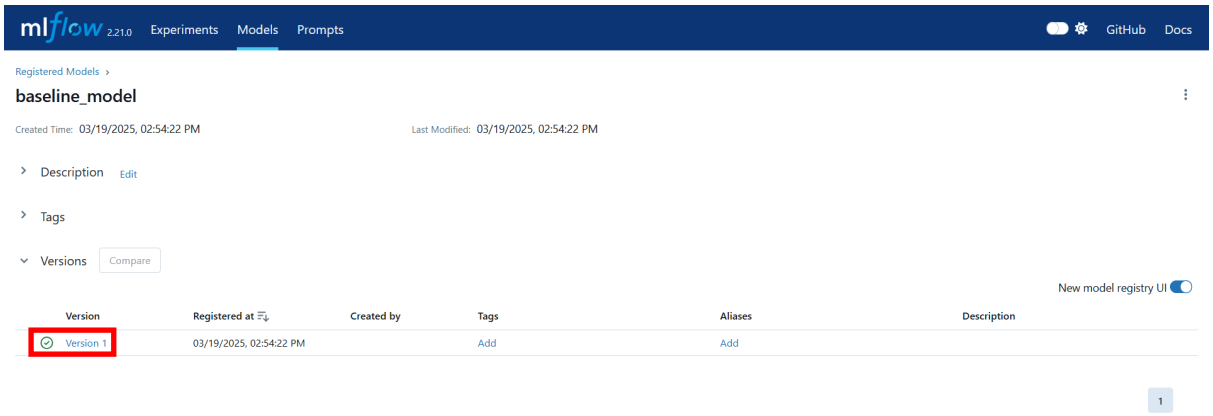
4. Note down the model uri printed out. In the screenshot below, the model uri is `runs:/c6dfb032b6b147568a7c9da318b81731/baseline_model`



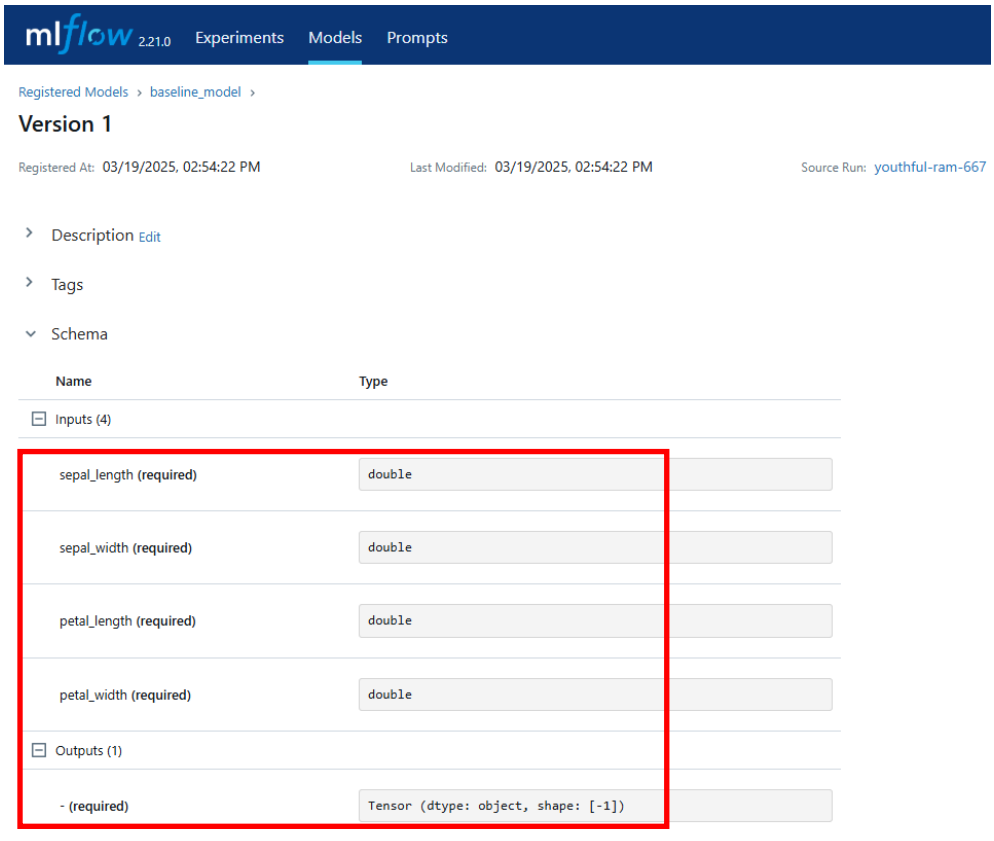
5. Open your web browser and navigate to http://localhost:9080 to view the MLflow user interface. Under the "Models" tab, look for "baseline_model" and click on it.



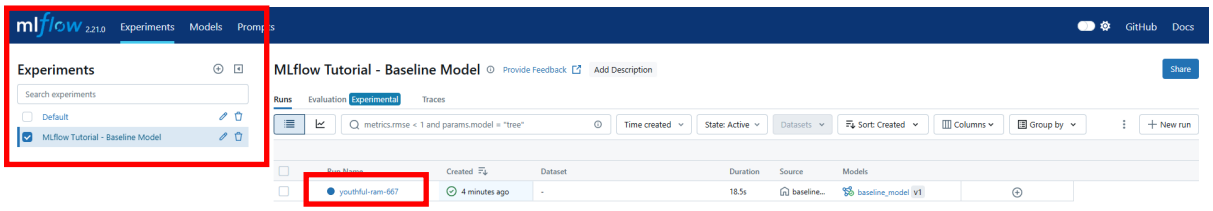
6. In the model web page, click on “Version 1”.



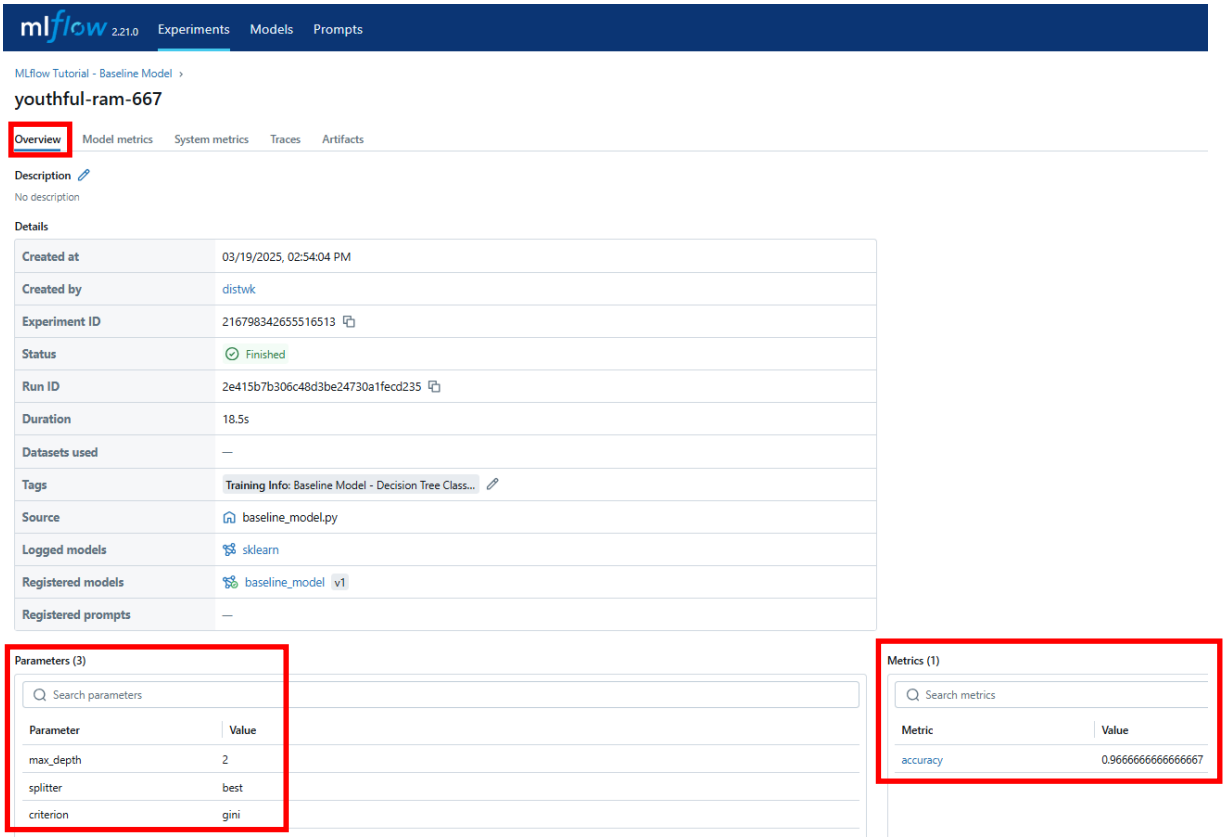
7. In the model version web page, expand the “Inputs” and “Outputs” section and verify that the input features are correct.



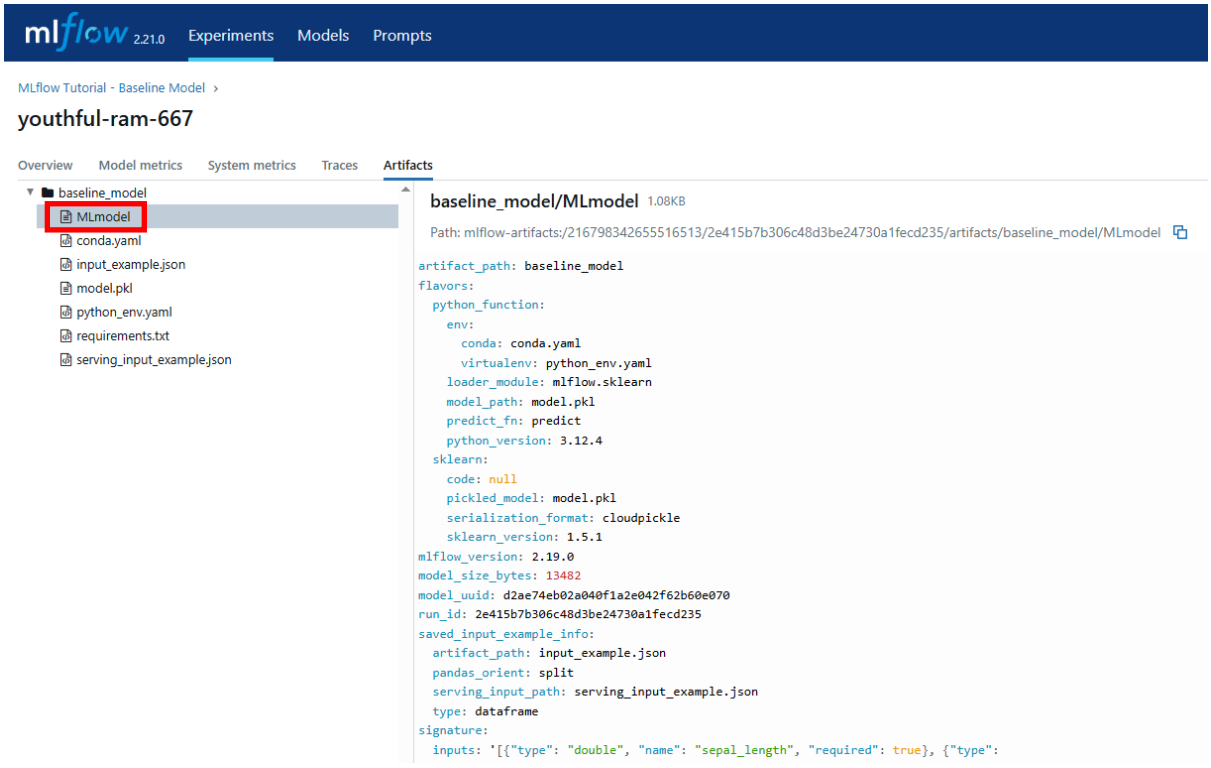
8. Click on the “Experiments” tab and then click on “MLflow Tutorial - Baseline Model”

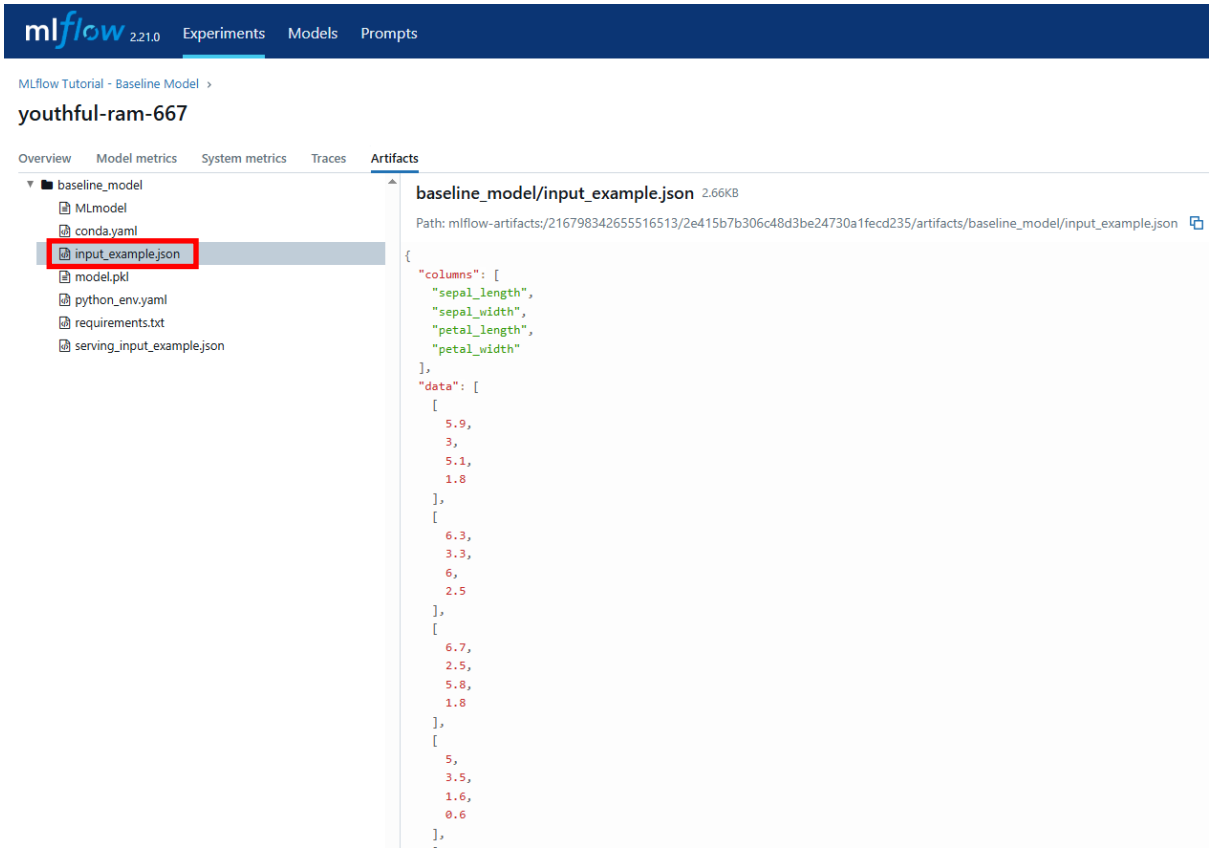


9. Click on the run link to view the run parameters and metric under the “Overview” tab. Note that the metric represents the testing accuracy of the model.



10. Click on the “Artifacts” tab to view the run’s artifacts. In particular, examine “MLmodel” and “input_example.json”.





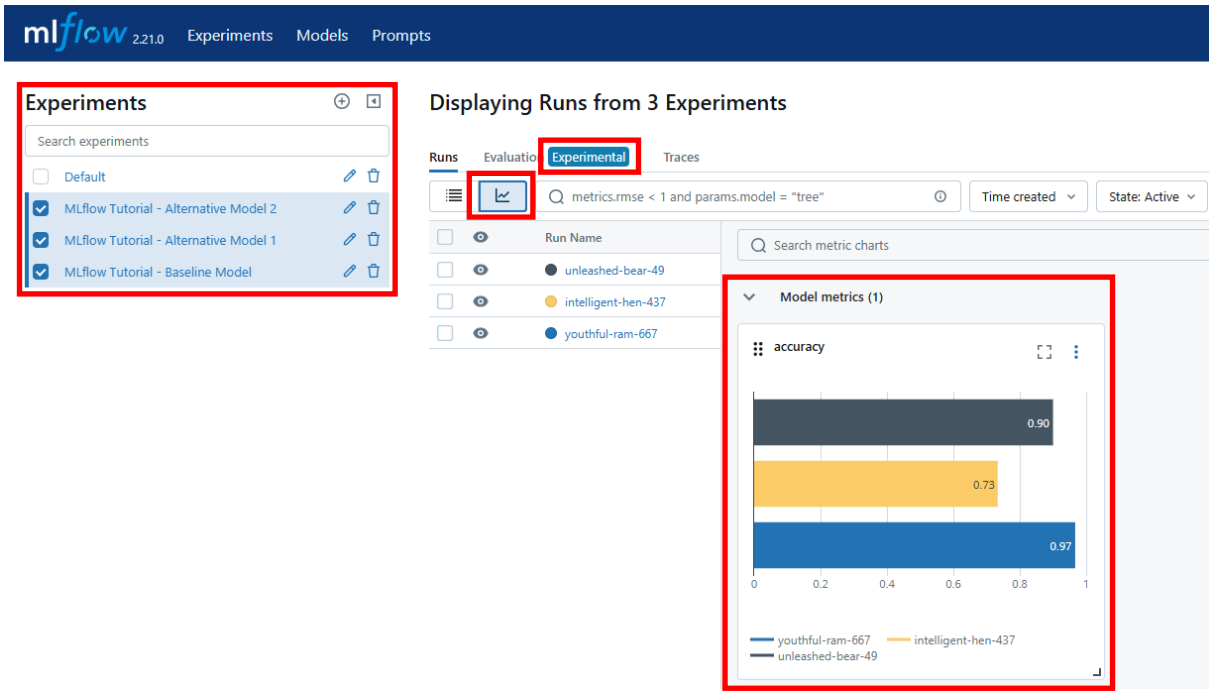
11. When you are done, navigate back to the home page of the MLflow user interface.

12. Run “alternative_model_1.py” and “alternative_model_2.py”. Remember to change the port number on line 38, if necessary.

```
python alternative_model_1.py
python alternative_model_2.py
```

13. Return back to MLflow user interface. Under the "Experiments > Experimental" tab, click on "Chart view" to compare the accuracy metric of the three models.

    Explain your observation and state which is the best model. In particular, why do you think the baseline model and alternative model 2 have similar accuracy metrics but alternative model 1 has a significantly lower accuracy metrics?



14. We will now load back "Baseline Model" and use it for inference. Open the Python source file "baseline_model_predict.py".

    Look for Line 21 and 32, then replace the model uri with the uri that you have noted down in step (4).

    Save the source file.

15. Run "baseline_model_predict.py" and observe the output:

```
python baseline_model_predict.py
```

16. In particular, observe the parameters of the model:

```
PROBLEMS  OUTPUT  TERMINAL  PORTS  POSTMAN CONSOLE  DEBUG CONSOLE

(.venv) C:\Users\distwk\workspace\src>python baseline_model_predict.py
0.9666666666666667
Model Parameters:
ccp_alpha : 0.0
class_weight : None
criterion : gini
max_depth : 2
max_features : None
max_leaf_nodes : None
min_impurity_decrease : 0.0
min_samples_leaf : 1
min_samples_split : 2
min_weight_fraction_leaf : 0.0
monotonic_cst : None
random_state : None
splitter : best
5
4
[ 2.44999999 -2.          1.75       -2.         -2.        ]
[ 2 -2  3 -2 -2]
```

-- End of Lab --